

TP01 – Premiers shaders

 Nous allons réaliser dans ce TP un « color picker » en OpenGL, autrement appelé en français, un sélecteur de couleur.



Exercice 0 Préparation de votre environnement de travail

0.1 Mise en place de la solution

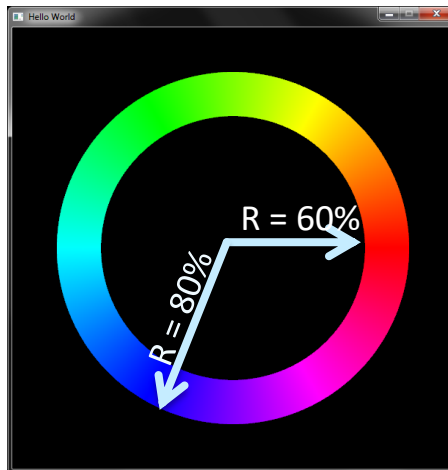
-  Reprenez les étapes vues lors du TP00 pour mettre en place votre solution Visual Studio.
 -  Vous pouvez pour cela copier-coller le répertoire **TP00** en **TP01** et renommer les fichiers **TP00.sln**, **TP00\TP00.vcxproj** et **TP00\TP00.vcxproj.filters** respectivement en **TP01.sln**, **TP01\TP01.vcxproj** et **TP01.vcxproj.filters**.
 -  Il vous faudra éditer à l'aide d'un éditeur de texte standard ces trois fichiers et remplacer toutes les références à « **TP00** » par « **TP01** ».
-  Nous utiliserons pour ce TP les classes de chargement / compilation de shader vu en cours.
 -  Votre professeur préféré en a fait une bibliothèque disponible dans le répertoire **ShaderPP-install** du fichier **Ressources.zip**.
 -  Vous devez donc paramétrer votre solution pour intégrer cette bibliothèque. Pour cela effectuez les configurations suivantes pour votre solution :
 -  Répertoire include supplémentaire :
\$(SolutionDir)\..\..\Ressources\ShaderPP-install\include
 -  Répertoire de bibliothèques supplémentaire :
\$(SolutionDir)\..\..\Ressources\ShaderPP-install\lib\\$(PlatformShortName)\\$(Configuration)
 -  Dépendance supplémentaire de l'éditeur de liens :
ShaderPP.lib
 -  L'utilisation de ces classes se fera par l'inclusion des fichiers **<Shader.h>**, **<ShaderProgram.h>** et **<ShaderException.h>** en fonction de vos besoins.

Exercice 1 Création géométrique du « color picker »

🖱️ Cette partie permet d’afficher les éléments graphiques du sélecteur de couleur.

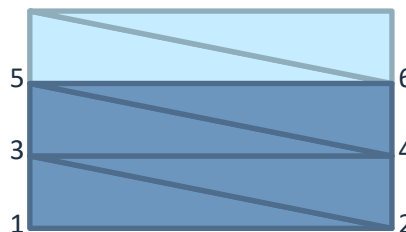
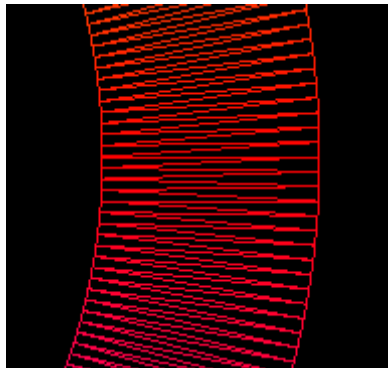
1.1 Le cercle des couleurs

Dimensions :



Construction :

Ce cercle est composé de petits triangles collés les uns aux autres, deux triangles par degré



Le type de construction de primitive OpenGL qui correspond à ce mode de construction est le mode `GL_TRIANGLE_STRIP`. (RTFM : https://www.opengl.org/wiki/Primitive#Triangle_primitives)

- 📖 Compte tenu de ces informations, faites en sorte que votre programme affiche exactement ce cercle.
- ➡️ Vous devrez implémenter le vertex shader et le fragment shader adéquat, puis réaliser le tracé avec la fonction adéquate.

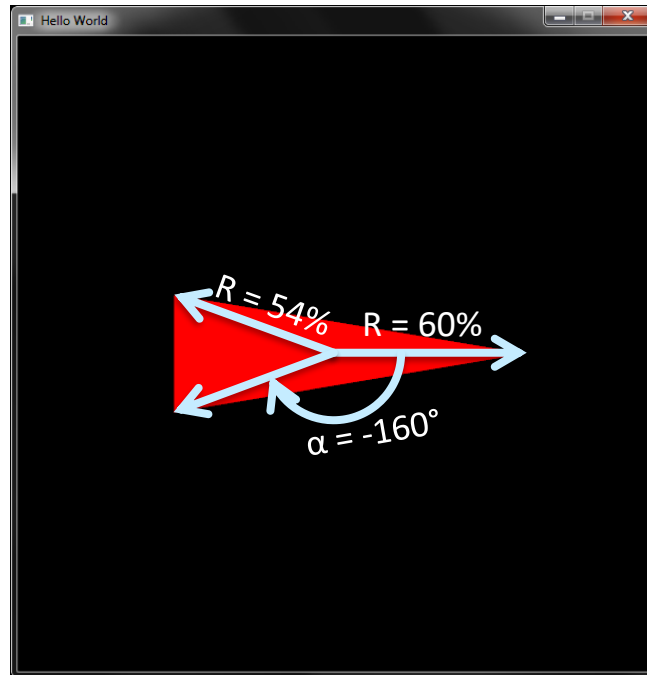


ATTENTION !!!

Les compilateurs GLSL sont très capricieux ! **NE COPIEZ COLLEZ JAMAIS DU CODE VERS UN FICHIER GLSL !** Vous risquez de copier des caractères de contrôles, invisibles, qui mettront en échec le compilateur sans message d’erreur pertinent.

1.2 Le curseur

Dimensions :



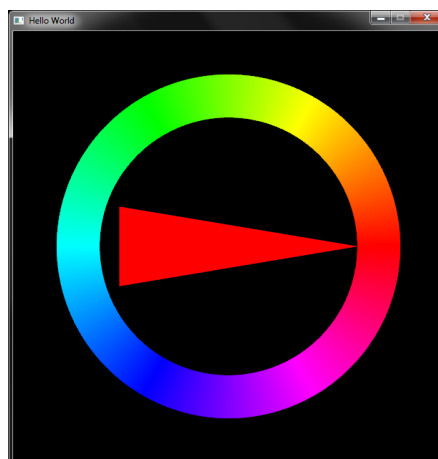
Construction :

Ce triangle aura une couleur unie, paramétrée par la couleur sélectionnée. Son orientation est dépendante de la teinte de la couleur.

- ☞ Compte tenu de ces informations, faites en sorte que votre programme affiche exactement ce triangle.
- 💻 Vous devrez implémenter le vertex shader et le fragment shader adéquat, puis réaliser le tracé avec la fonction adéquate.
- ➡ Pour le moment, affichez ce triangle en rouge avec une orientation de 0° . Vous devez cependant préparer le paramétrage futur de la couleur et de l'orientation.

1.3 Résultat final de l'exercice 1

En activant les deux affichages, vous devriez obtenir l'image ci-dessous.

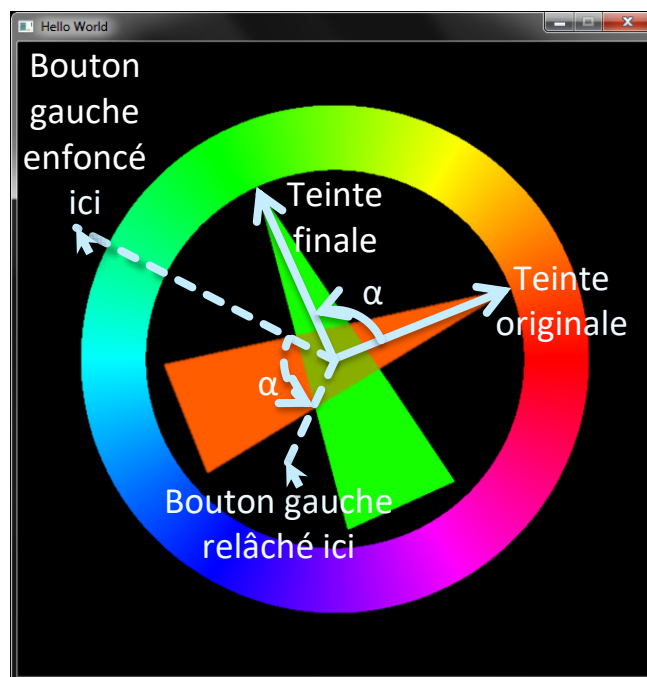


Exercice 2 Interactions

Il y a trois interactions à implémenter, une par composante HSV de la couleur.

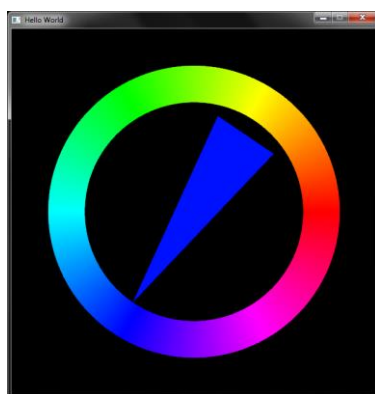
- ☞ Implémentez ces interactions selon leurs spécifications indiquées ci-dessous.
 - ☞ La rotation du curseur doit se faire dans le vertex shader, de préférence en utilisant un calcul matriciel.
 - ☞ Les calculs de conversion HSV $\rightarrow$ RGB peuvent se faire dans le code CPU (utiliser pour cela la classe **CColor** des exemples de cours).

H : Lorsque le bouton gauche de la souris est enfoncé, le curseur pivote relativement à l'angle formé entre le centre du cercle et la position de la souris à l'origine du clic d'une part, et le centre du cercle et la position courante de la souris d'autre part :

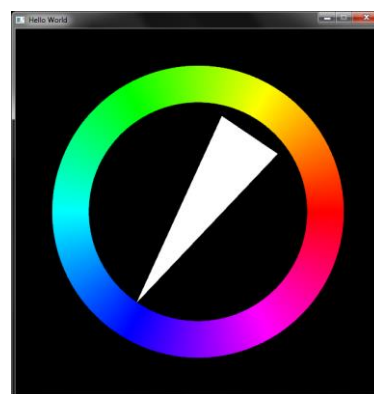


$$H_{\text{final}} = H_{\text{initial}} + \alpha, \alpha \text{ étant l'angle obtenu par le mouvement de la souris}$$

S : La saturation est pilotée par la molette de la souris. Un mouvement de la molette vers le bas diminue la saturation alors que le mouvement inverse l'augmente.

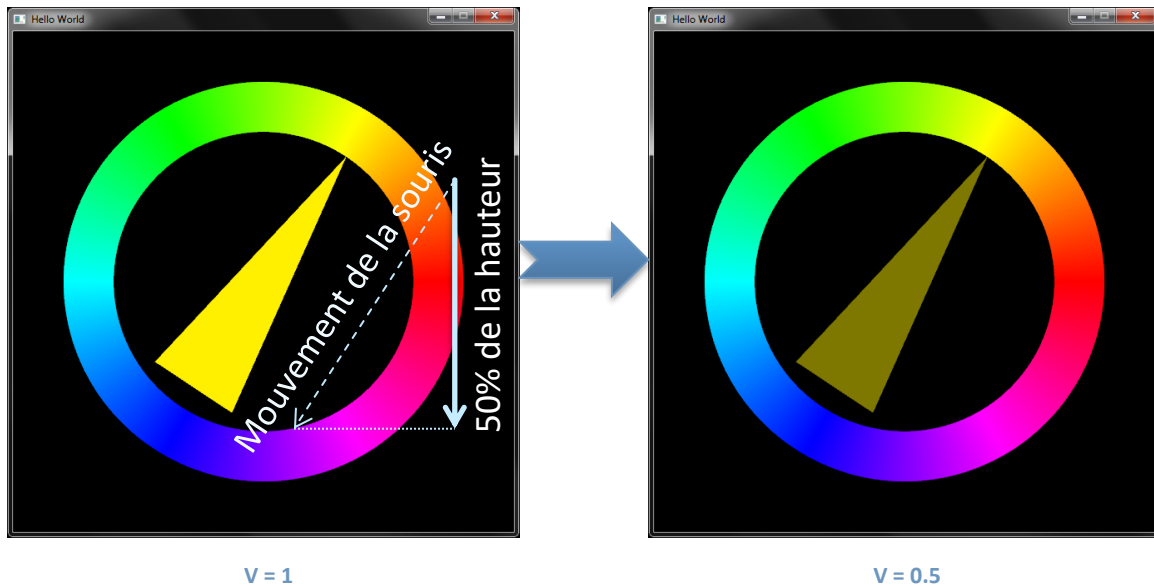


$S = 1$



$S = 0$

V : La valeur (luminosité) de la couleur suit les mouvements verticaux de la souris lorsque le bouton droit est enfoncé :

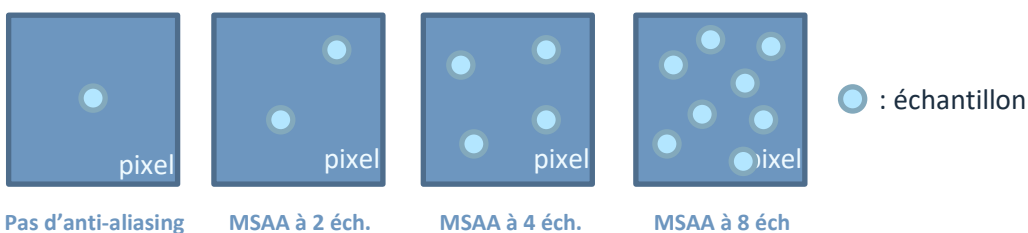


Exercice 3 Si vous avez le temps

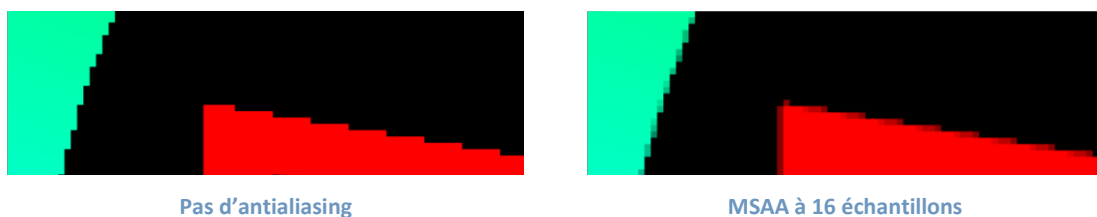
3.1 Informations sur l'anti-aliasing

En observant attentivement les contours du cercle et du triangle, vous pouvez observer un effet de « marches d'escalier ». Il est possible d'activer une fonctionnalité d'OpenGL qui permet de résoudre cet effet disgracieux : le **multisampling** (ou MSAA pour MultiSamplingAntiAliasing). Cette technologie consiste à calculer plusieurs échantillons pour les fragments des bordures de polygones, puis à moyenner ces échantillons entre eux pour calculer le fragment final. Les différents échantillons sont répartis dans le même pixel de façon à ce que si le pixel est « à cheval » entre un polygone et un autre (ou le fond) la couleur résultante de la moyenne mélange judicieusement les deux couleurs.

Exemple de répartition des échantillons pour un pixel :



Effet de l'anti-aliasing :



3.2 Mise en place de l'anti-aliasing

Mettre en place cette fonctionnalité est extrêmement facile :



Créez la fenêtre avec la capacité de gérer le multisampling



Utiliser l'appel suivant avant la création de la fenêtre :

```
glfwWindowHint(GLFW_SAMPLES, 16); //ici 16 échantillons seront utilisés
```



Activer le multisampling



Dans le contexte OpenGL de la fenêtre, appeler la fonction suivante :

```
glEnable(GL_MULTISAMPLE);
```